A

Mini Project Report

On

KVSR_Dashboard using Python

Submitted to DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING(AIML/AI)

Submitted By

G.Jaswanth	**(24FH1A3333)**
P.Vara Prasad	**(24FH1A3357)**
M.Vijay Kumar	**(24FH1A3352)**
Rs Hemanth	**(24FH1A3359)**
AJITH KUMAR REDDY	**(24FH1A3361)**

Under the Esteemed guidance of

Dr. B.MAHESH

Associate Professor, Department of CSE

Department of CSE(AI & AIML)

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTUA, Anantapur)

2026-2027

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Accredited by NBA,Approved by AICTE NEW DELHI, Affiliated to JNTUA, Anantapur)

Department of CSE(AI/AIML)

CERTIFICATE

This is to certify that the project entitled **"KVSR_Dashboard using Python"** is a bonafide work carried out by

G.Jaswanth	**(24FH1A3333)**
P.Vara Prasad	**(24FH1A3357)**
M.Vijay Kumar	**(24FH1A3352)**
Rs Hemanth	**(24FH1A3359)**
AJITH KUMAR REDDY	**(24FH1A3361)**

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE(AI/AIML)** from Dr.K.V.Subba Reddy Institute of Technology, Kurnool, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

Guide **Head of the Department**

DECLARATION

This is to certify that the work reported in the present Mini project entitled **"KVSR_Dashboard using Python"** is a record of bonafide work done by us in the Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology, Kurnool. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

G.Jaswanth	**24FH1A3333**
P.Vara Prasad	**24FH1A3357**
M.Vijay Kumar	**24FH1A3352**
Rs Hemanth	**24FH1A3359**
AJITH KUMAR REDDY	**24FH1A3361**

ACKNOWLEDGMENT

We are extremely grateful to **Dr. K Sree Latha**, Principal and **H. Ateeq Ahmed**, HOD, **Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology** for their constant support.

We are extremely thankful to **Dr. B.Mahesh**, Associate Professor, **Department of CSE**, Internal Guide, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

G.Jaswanth **(24FH1A3333)**
P.Vara Prasad **(24FH1A3357)**
M.Vijay Kumar **(24FH1A3352)**
Rs Hemanth **(24FH1A3359)**
AJITH KUMAR REDDY **(24FH1A3361)**

Report guide / Table of contents

This document follows the 14-section structure supplied in visualization-project-index.pdf. The section numbering has been normalized for consistency. Descriptions and numerical results are grounded in the uploaded Python implementation rather than an imagined version of the project.

01 Abstract

02 Introduction (background, motivation, problem, objectives, scope)

03 Literature Survey / Existing System

04 Technologies and Tools Used

05 System Requirements

06 Dataset Description

07 Implementation

08 Data Visualization

09 Results and Discussion

10 Advantages and Limitations

11 Conclusion

12 Future Enhancement

13 References

14 Appendix

Reading the report

Actual features: Seven dashboard pages, five global smart filters, searchable student directory, KPI cards, interactive Plotly charts, risk summaries, and CSV download actions.
Evidence type: Source-code analysis and recreated charts from seeded demo data.
Four Android Chrome screenshots supplied for this edition appear in Appendix C.

Data integrity: The application explicitly states that its attendance figures and most student records are demonstration data. The third-year AI & ML roster contains user-supplied names and IDs; attendance and sections for that roster are simulated as well.

Privacy: This report deliberately avoids reproducing student names and identifiers; charts contain aggregate counts only.

01 Abstract

The KVSR Smart Campus Intelligence dashboard is a single-file, Python-based web application for exploring student distribution and attendance patterns across academic departments and years. Developed with Streamlit, Pandas and Plotly, the application transforms student-level tabular records into executive metrics, interactive visualizations, risk classifications, searchable tables and downloadable summaries. It is designed as an academic visualization prototype rather than an operational student information system.

The supplied code generates 1,441 demonstration records across eight department labels and four academic-year categories. It reports a simulated mean attendance of 80.11%, with 506 records below the selected 75% attendance benchmark. These values are useful for explaining analytical workflows but must not be represented as verified institutional outcomes.

Keywords

Data visualization; Streamlit; Python; Pandas; Plotly; campus analytics; dashboard; attendance classification; interactive filtering; synthetic data.

02 Introduction

2.1 Background

Educational administrators and academic teams often need clear summaries of student strength, departmental distribution and attendance status. Spreadsheets can store these records, but repeated manual filtering and chart preparation make routine comparisons cumbersome. The supplied project brings these operations into a single Streamlit interface with synchronized global filters and visual summaries.

2.2 Motivation

The central motivation is to create a readable, exploratory dashboard that helps a viewer identify differences between departments and academic years, inspect attendance classifications, and export selected records. The implementation emphasizes visually distinct KPI cards, high-contrast navigation, neatly spaced charts and structured risk reporting.

2.3 Problem Statement

Design a Python-based interactive student analytics dashboard that can display the distribution of student records, compute attendance aggregates, highlight values below a configured benchmark, and allow non-programmer users to explore results through department, year, gender, section and attendance-band filters.

2.4 Objectives of the Project

- Consolidate student-level demonstration records into an in-memory Pandas DataFrame.
- Present executive counts and attendance metrics without requiring command-line analysis.
- Visualize department strength, year composition, gender shares and attendance patterns.
- Identify and display students whose simulated attendance falls below 75%.
- Enable interactive selection, drill-down, text search and CSV exports.
- Provide a consistent web UI with responsive layout adjustments for narrow screens.

2.5 Scope of the Project

The delivered code covers student profiles, department- and year-level comparisons, attendance analysis, visualization and CSV extraction. It does not implement a secure login portal, live institutional database, attendance input workflow, student record editing or authenticated access. The interface runs as a Streamlit web application and can be viewed in a compatible mobile browser when deployed accessibly; it is not a native Android APK.

03 Literature Survey / Existing System

3.1 Existing Data Visualization Tools

Common approaches to campus data exploration include spreadsheet pivot tables, static printed reports, business intelligence dashboards and custom web portals. In comparison, a Python/Streamlit prototype offers a compact way to combine data transformations, Python analytics and browser-based output in one program. This is a conceptual comparison, not a performance experiment or evidence of a particular college's current system.

Approach	Typical capability	Trade-off for this project
Spreadsheet	Manual analysis, filters, pivot charts	Easy to edit, less unified navigation
Static report	Portable tables and figures	No live interactive drill-down
BI dashboard	Rich reporting and connectivity	Additional platform and data configuration
Streamlit + Plotly	Python-driven web UI and interactive charts	Needs Python runtime and data governance

3.2 Limitations of Existing System

In a manual reporting workflow, department statistics may require repeated pivot-table operations, attendance-risk lists can be difficult to maintain, and cross-filtered chart creation may be time-consuming. Such limitations motivate this demonstration application; they have not been benchmarked against a real existing software installation.

3.3 Proposed System

The proposed implementation places filtering and aggregation in Pandas, generates interactive Plotly figures, and presents all pages inside a shared Streamlit sidebar framework. It keeps charts responsive to the active DataFrame subset and provides explicit CSV download buttons for selected views.

04 Technologies and Tools Used

Technology	Role in source code	Status
Python	Control flow, functions, random generator and timestamp	Implemented
Streamlit	Page config, sidebar, widgets, tabs, metrics, tables and downloads	Implemented
Pandas	DataFrame, groupby, pivot and crosstab-like summaries	Implemented
Plotly Express	Bar, pie/donut, sunburst and box charts	Implemented
Plotly Graph Objects	Attendance heatmap	Implemented
NumPy	Mentioned in supplied index but not directly imported	Not used directly

Matplotlib / Seaborn	Mentioned in supplied index but not used by application	Not used in app

The code header specifies dependency ranges: streamlit >= 1.40 and < 2; plotly >= 5.24 and < 7; pandas >= 2.2 and < 3. Built-in random and datetime modules require no extra installation. The static figures embedded in this report were regenerated separately for documentation; this does not mean the dashboard itself uses Matplotlib.

05 System Requirements

5.1 Hardware Requirements

For classroom demonstration, a modern laptop or desktop with a dual-core CPU, approximately 4 GB RAM or more, and ordinary free disk space should be adequate for a small in-memory example. These are practical recommendations, not hardware requirements enforced by the program. Viewer devices need only a modern browser and network access to the running Streamlit service.

5.2 Software Requirements

Component	Recommended / source-specified configuration
Operating system	Windows, Linux or macOS; Android browser as a viewer
Python	A recent Python 3 version supported by chosen Streamlit/Pandas versions
Python packages	streamlit>=1.40,<2 ; plotly>=5.24,<7 ; pandas>=2.2,<3
Browser	Recent Chrome, Edge, Firefox or Safari
Data connection	Not required for generation of current demo records

Installation and launch

```
python -m pip install "streamlit>=1.40,<2" "plotly>=5.24,<7" "pandas>=2.2,<3"
python -m streamlit run "KVSR_Dashboard_Advanced-1(3).py"
```

Run the command from the folder containing the file. The browser URL is ordinarily shown by Streamlit after startup. The uploaded file name is used above; the comment inside the Python source uses a different example file name and should not be copied without checking the actual saved name.

06 Dataset Description

6.1 Dataset Source

The active dataset is created in the create_data() function inside the Python file. It is not loaded from an external CSV, Excel workbook, network API or database. The function seeds Python's random generator to make the demonstration dataset repeatable under the same execution environment. It includes a user-provided third-year AI & ML student-name/ID list; other names and IDs are generated programmatically. Attendance and section assignments are simulated throughout.

6.2 Dataset Attributes and Data Types

Field	Indicative type	Meaning
Student ID	Text	Unique identifier or generated KVSR identifier

Name	Text	Student display name
Department	Categorical text	Department shorthand
Department Name	Text	Expanded department name
Year	Categorical text	1st, 2nd, 3rd or 4th year
Section	Categorical text	A or B; simulated in source
Gender	Categorical text	Male/Female labels in source
Attendance	Decimal number	Simulated percent attendance

6.3 Data Coverage

The program uses CSE, CSE (AI & ML), CSE (DS), ECE, EEE, CIVIL, MECH and MBA department labels. Its generation loop omits MBA records for third and fourth year, while other departments cover all four years. Department counts depend on the seeded sample logic; the report records the resulting dataset generated from the supplied source.

Dataset check	Result (demo)
Total student rows	1,441
Unique student IDs	1,441
Department labels	8
Academic-year labels	4
Average simulated attendance	80.11%
Below 75%	506 rows
Below 65%	129 rows

6.4 Data Preprocessing

The workflow constructs dictionaries for each student row and converts the result to a DataFrame. The dashboard then derives filtered views using equality conditions, attendance thresholds and Pandas aggregations. Year ordering is explicitly controlled for some charts through a categorical sequence.

6.5 Data Cleaning and Validation

The supplied application performs direct filtering and aggregation but contains no comprehensive input validation, missing-value imputation pipeline or data repair interface. For this report, the standalone data-generator output was checked for unique student IDs, attendance between 0 and 100, mutually exclusive attendance bands and the MBA year restriction. These checks passed for the generated example; they do not validate external real-world records.

07 Implementation

7.1 Project Structure

All application functionality is contained in one Python file, KVSR_Dashboard_Advanced-1(3).py. It defines page configuration, extensive custom CSS, a cached dataset generator, chart/KPI helpers, sidebar filters and seven mutually exclusive main-menu views. No additional project-specific dataset file is necessary to run the current demonstration.

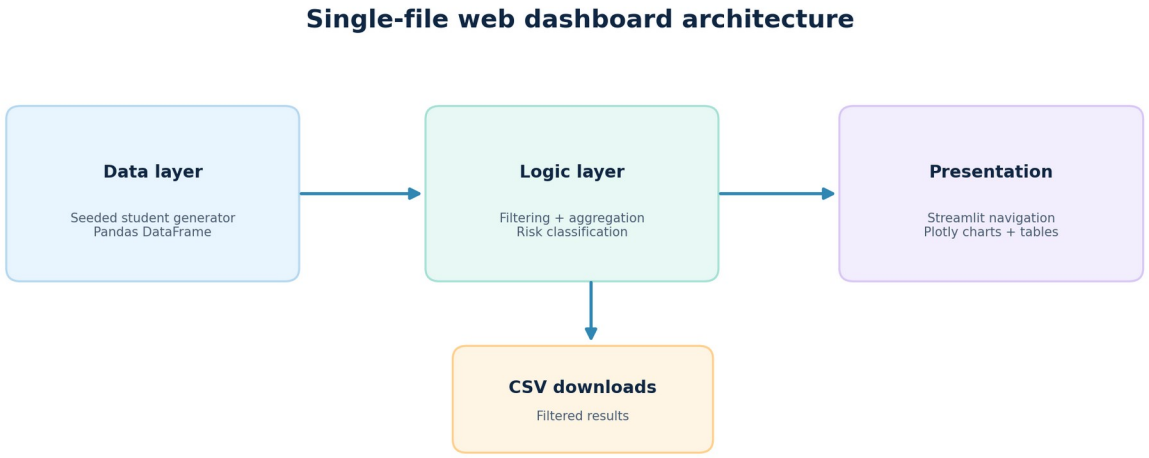


Figure 1. Logical pipeline reconstructed from the actual single-file code.

7.2 Data Loading

The @st.cache_data decorator caches create_data(), avoiding repeat regeneration of the same demo DataFrame during ordinary Streamlit reruns. After df = create_data(), all views operate on df or on a filtered copy called f. There is no persistence layer and no database read query.

7.3 Data Preprocessing and Analytical Rules

An active filter first creates f = df.copy() and applies zero or more conditions for department, year, gender, section and attendance band. From f, the program computes student count, department count, mean attendance and attendance status counts. The risk rule is Attendance < 75; the excellent rule is Attendance >= 90; the intermediate range is 75 to below 90.

```
at_risk = count(attendance < 75)
excellent = count(attendance >= 90)
good_range = count(75 <= attendance < 90)
average_attendance = mean(attendance)
```

7.4 Visualization Development

Visualization helpers standardize the color palette, title, margins, legend and chart height. The plot() helper increments an internal chart counter and supplies a unique Streamlit widget key to each Plotly chart. This specifically prevents duplicate element-ID collisions when several similar charts appear on a page. Plotly mode bar features include a PNG image export option.

7.5 Streamlit Interface Development

The design uses a blue-to-cyan header, a dark sidebar, white KPI cards, light page backgrounds, accent borders, styled select widgets and a responsive CSS rule for narrower screens. Navigation is performed with a sidebar radio control, not separate Python multipage scripts.

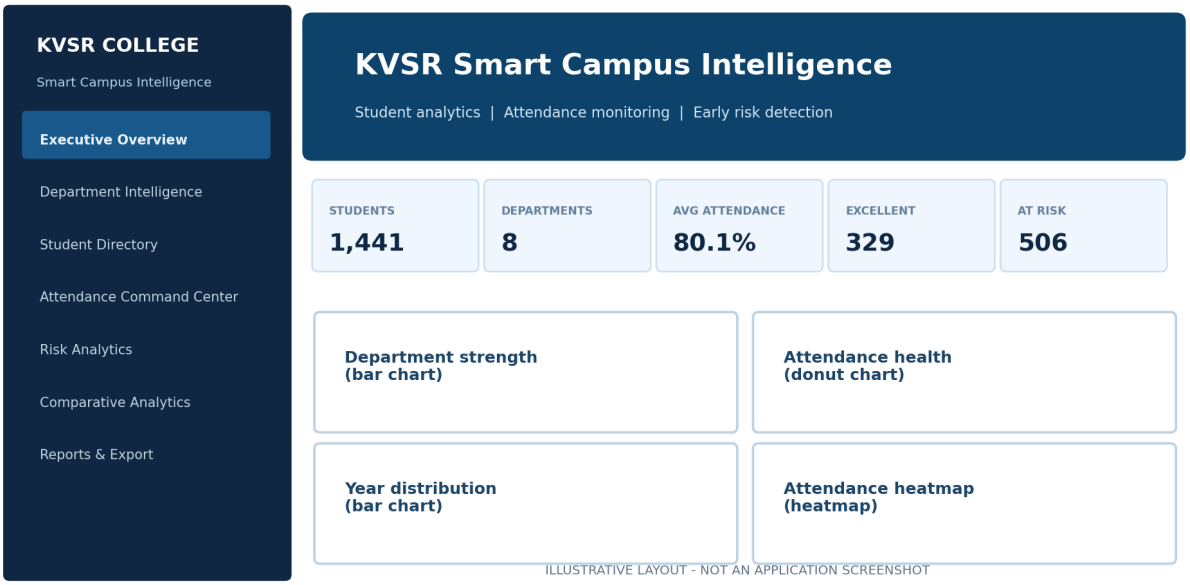


Figure 2. Conceptual interface reconstruction from the source UI - not a live screenshot.

7.6 Interactive Filters

Control	Options / behavior
Department	All Departments or one of eight department labels
Academic Year	All Years, 1st, 2nd, 3rd, 4th
Gender	All, Male, Female
Section	All, A, B
Attendance Band	All, Excellent >=90, Good 75-<90, Risk <75
Directory search	Case-insensitive literal substring match on name or student ID
Chart exploration	Plotly hover; sunburst click-to-drill into department/year

7.7 Dashboard Integration / Seven Pages

Dashboard section	Main functionality
1. Executive Overview	KPI cards, department bar, attendance donut, year bar, gender donut, sunburst, heatmap, box plot
2. Department Intelligence	Department summary table, strength and attendance bars, leading average insight
3. Student Directory	Searchable student table, gender counts, average attendance, filtered CSV download
4. Attendance Command Center	Attendance summary KPIs, department attendance, distribution bars and low-attendance queue
5. Risk Analytics	Below-75 and below-65 metrics, department risk chart, year-wise risk donut, risk table
6. Comparative Analytics	Grouped bars comparing department-by-year strength or attendance; pivot table
7. Reports & Export	Department report table, two CSV download actions, average-attendance bar

08 Data Visualization

The following figures were created from the code's generated demonstration data and use the same analytical groupings as the dashboard. They are documentation figures, not screenshots from a running Streamlit process. All figures should be interpreted as demonstrations of functionality, not institutional performance.

8.1 Bar Chart - Department Distribution

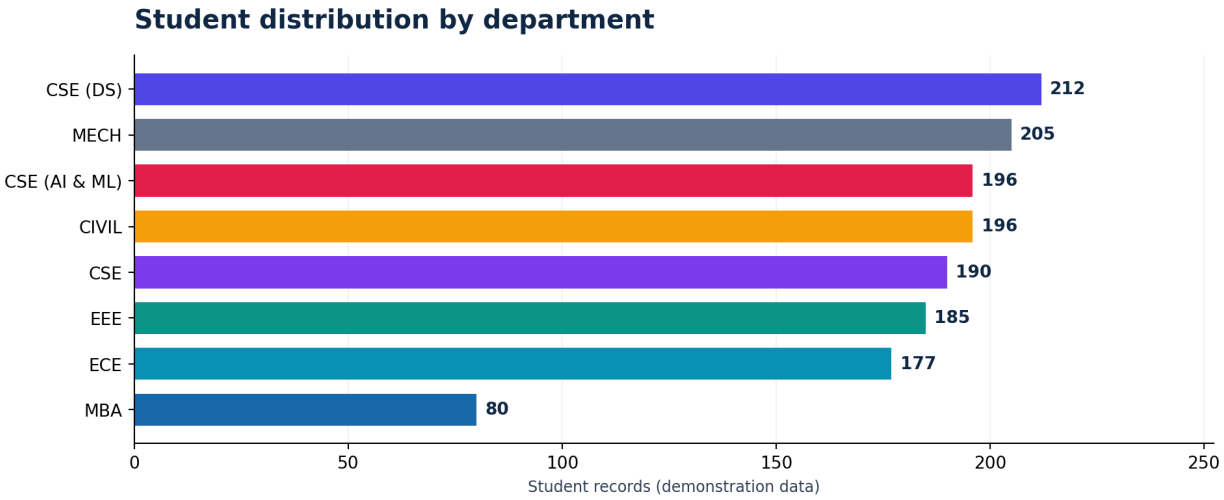


Figure 3. Department count distribution across 1,441 demo student rows.

The comparison highlights differences in generated department strength. CSE (DS) has 212 rows, MECH 205 rows and MBA 80 rows in the current seeded sample. These values depend on the dataset generator and should not be read as actual college enrolment.

8.2 Pie / Donut Chart - Attendance Health

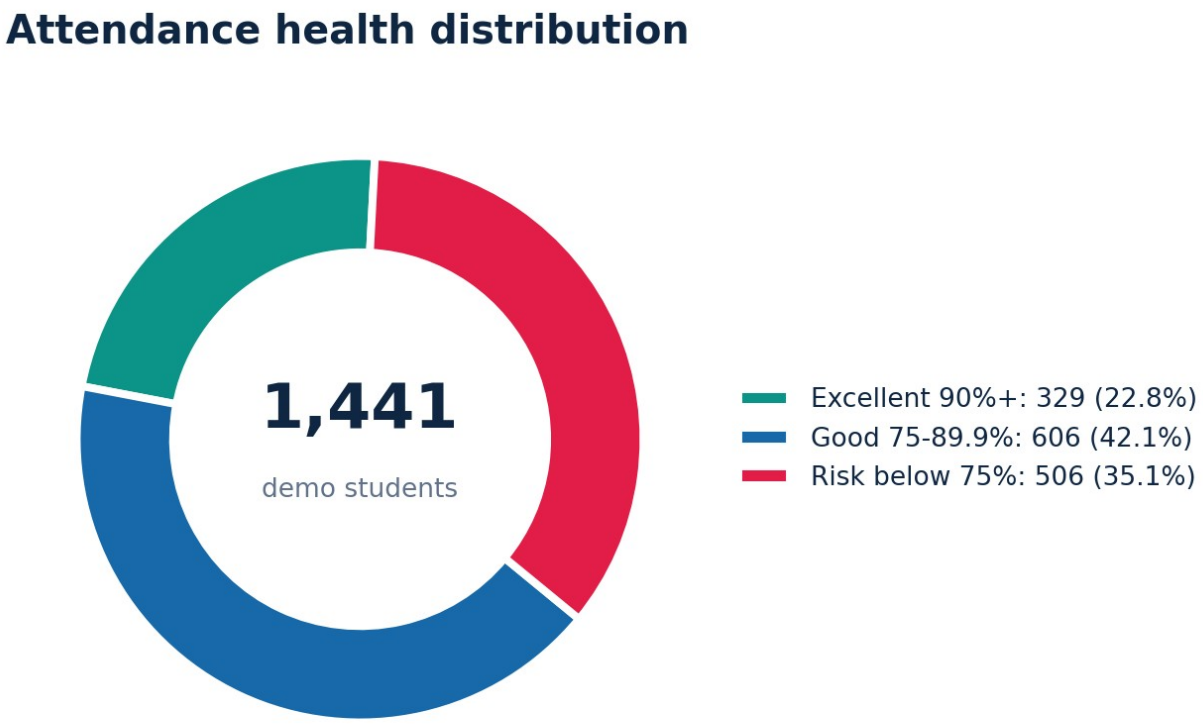


Figure 4. Simulated attendance categories, partitioning all records into mutually exclusive bands.

The demo output contains 329 records with attendance >=90%, 606 between 75% and below 90%, and 506 below 75%. The donut visualization expresses the relative size of these categories; the Streamlit app renders an interactive Plotly equivalent.

8.3 Bar Chart - Academic Year Distribution

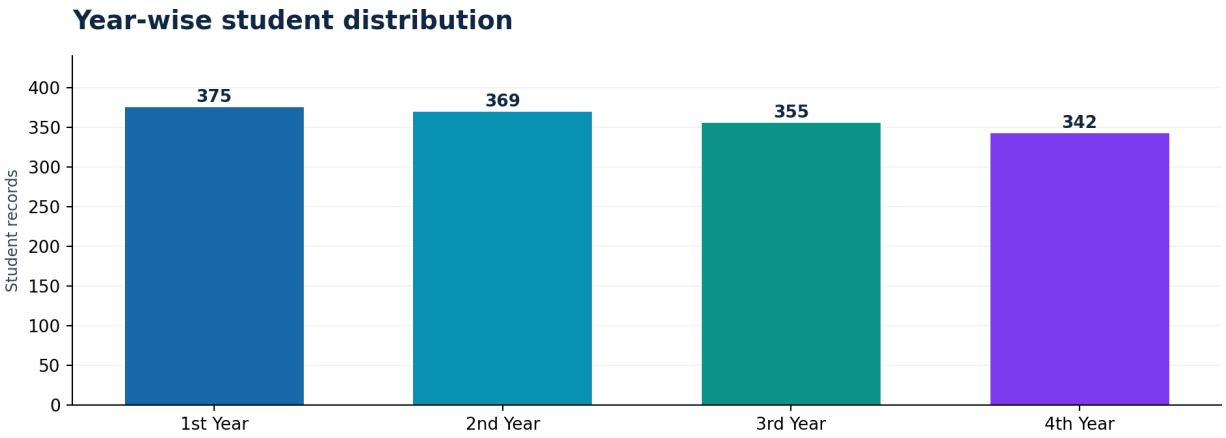


Figure 5. Demo student totals grouped by academic year.

Generated cohort counts are 375 (1st), 369 (2nd), 355 (3rd) and 342 (4th). The 3rd and 4th year figures do not include MBA entries under the script's own cohort-generation rule.

8.4 Comparative Bar Chart - Mean Attendance

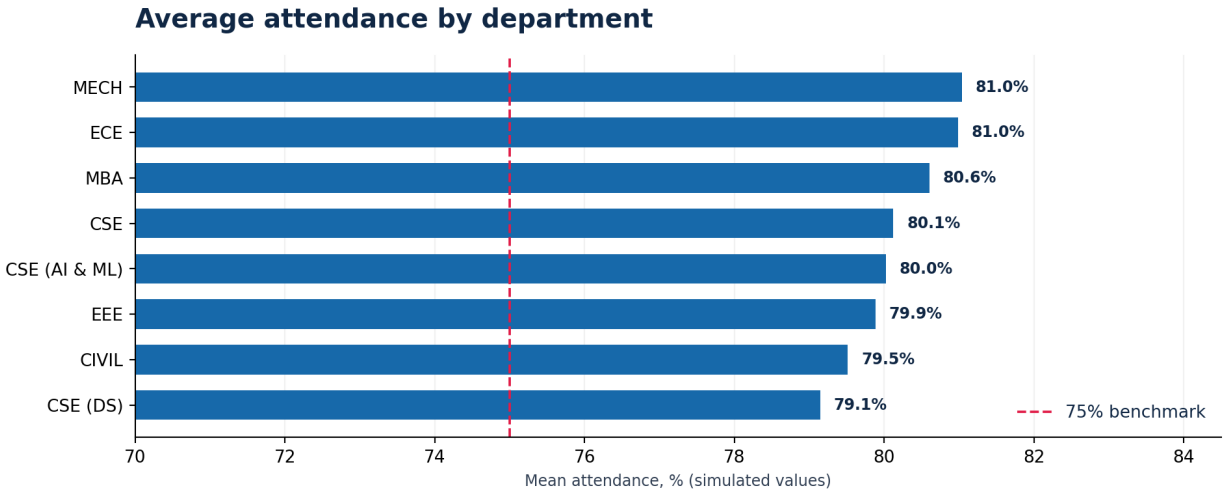


Figure 6. Mean simulated attendance by department with a reference benchmark.

Departmental simulated means are clustered near 79-81%. The vertical reference indicates the dashboard's attendance benchmark of 75%; a mean above 75% does not mean every student in that department meets the benchmark.

8.5 Heatmap - Attendance by Department and Year

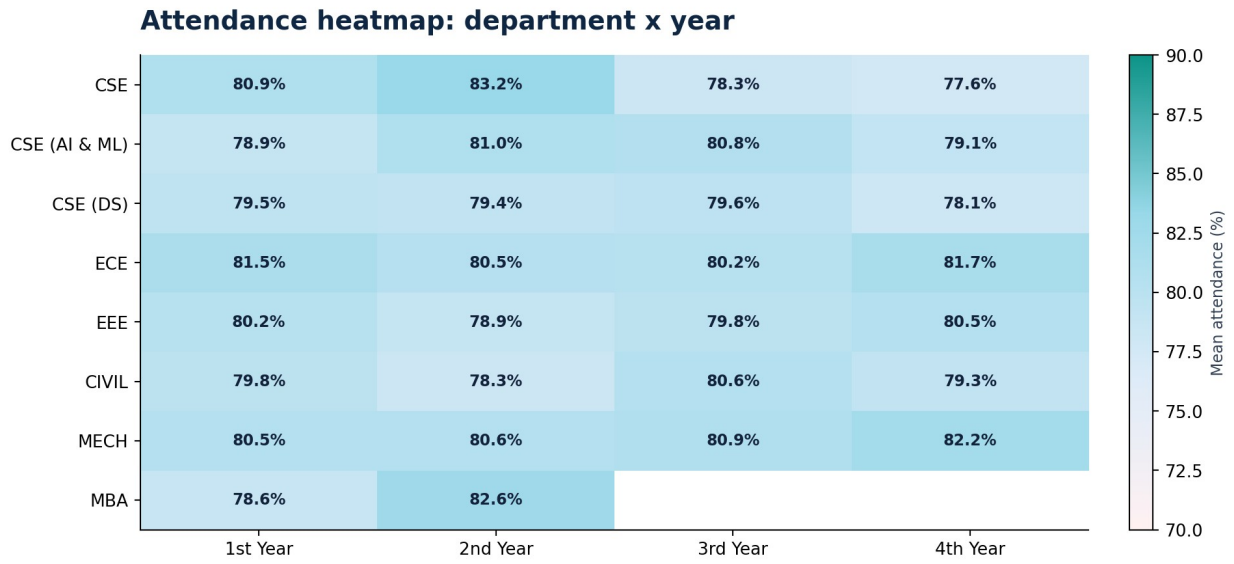


Figure 7. Pivoted department x year simulated attendance averages. Empty MBA cells represent absent groups, not 0%.

The Python dashboard builds this visual from a Pandas pivot_table and Plotly Graph Objects. It uses a fixed 0-100 color scale in the interactive interface; this report version uses a more focused color range for printed readability. Hover behavior is available only in the live Plotly chart.

8.6 Additional Visuals - Risk, Box and Sunburst

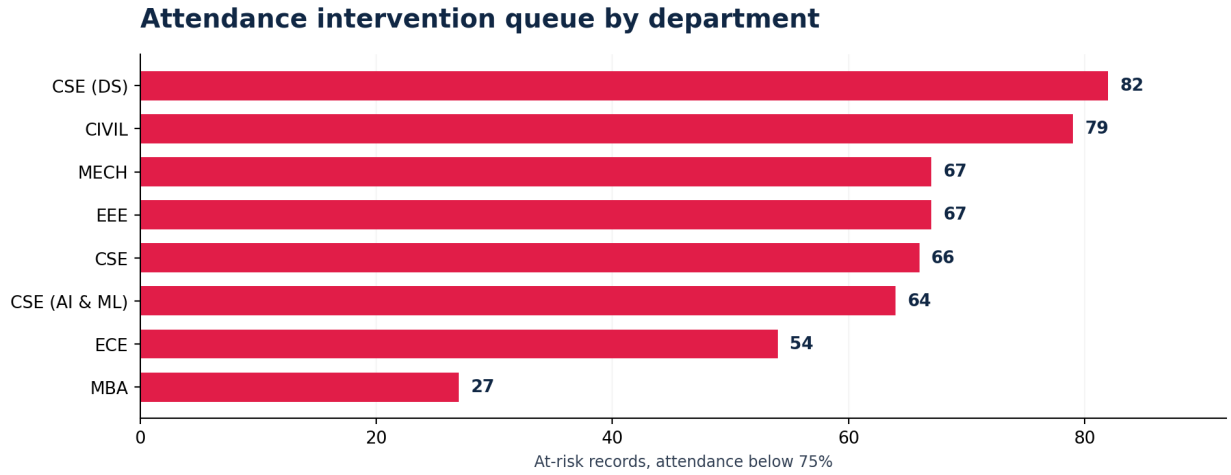


Figure 8. Counts below 75% by department, using the same risk rule as the application.

The source also implements a box plot for within-department attendance spread and an interactive department-to-year sunburst for student composition. The box plot uses a 75% horizontal reference line. Sunburst segments support click-to-drill, while the scatter plot, histogram and time-series line chart named in the generic index are not present in the uploaded application.

09 Results and Discussion

9.1 Dashboard Screenshots / Visual Evidence

Four captured Android Chrome screens are now included as Appendix C evidence.

Figure 2 remains a conceptual interface diagram, not a browser screenshot.

See Appendix C for four Android Chrome captures of the Streamlit dashboard.
Figures 3-8 visualize aggregate data regenerated from the source generator.

9.2 Visualization Results

Metric	Demo output	Interpretation
Student rows	1,441	Count of generated records
Departments	8	Distinct department codes
Average attendance	80.11%	Mean of simulated attendance field
Excellent >=90%	329 (22.8%)	Top attendance band
Good 75%-<90%	606 (42.1%)	Intermediate attendance band
Below 75%	506 (35.1%)	Records flagged for attention
Below 65%	129 (9.0%)	Critical subset defined in risk module

9.3 Analysis of Results

In the generated sample, 935 of 1,441 rows (64.9%) meet or exceed the 75% benchmark, while 506 fall below it. The most populous generated department is CSE (DS) with 212 records, but the department with the highest simulated average attendance is MECH at approximately 81.04%. Risk counts are influenced both by department size and the randomly assigned attendance percentages; cross-department comparisons require contextual care.

9.4 Key Insights

- Global filters propagate through the selected charts, metrics and tables rather than applying to one chart only.
- Risk analytics separately reports <65% and <75% thresholds, supporting different levels of visual attention.
- The box plot and heatmap reveal distribution and group-level variation that an overall mean can hide.
- The student directory can find a partial name or identifier and export the filtered subset.
- The numeric findings are properties of a seeded demonstration dataset, not verified student attendance reports.

9.5 Checks Performed for Documentation

The source file passed Python syntax parsing. Its create_data function was evaluated in isolation to regenerate the dataset for charts and tables, and validation checks passed for ID uniqueness, 0-100 attendance range, exact category partition and MBA year restrictions. These are source/data checks only; page-by-page Streamlit UI integration, actual browser responsiveness, access permissions and CSV click behavior were not end-to-end tested here.

10 Advantages and Limitations

10.1 Advantages

- All key analytics features are collected into a single Python script, simplifying classroom setup.
- Streamlit controls make exploration accessible without running separate Pandas commands.
- A broad visualization mix covers composition, comparison, variation and risk.
- Custom CSS makes the experience visually structured and adapts some elements to narrow screens.
- Unique plot keys reduce Streamlit duplicate-widget errors.

- CSV exports support follow-on analysis of the currently selected data.

10.2 Limitations

- The program does not read live attendance records or an institutional database.
- Most names and IDs are generated, and every attendance value and section is simulated.
- There is no login, access management, audit trail or consent workflow for personal data.
- Student entries are not edited or saved through the interface; there is no persistent data store.
- Attendance is a single percentage field, not an attendance history with dated sessions.
- A native mobile application package is not included; mobile access is browser-based.
- The interface provides interactive Plotly chart exploration, not fully linked cross-filtering between every chart.

11 Conclusion

The KVSR Smart Campus Intelligence project demonstrates how a single Python program can combine student-level example data, reusable filtering rules, Pandas grouping and Plotly interactivity into a cohesive Streamlit visualization dashboard. The seven-page application meets its prototype objective of supporting department comparisons, attendance interpretation, risk highlighting, directory search and CSV report generation. The demonstration results illustrate dashboard behavior rather than official outcomes, and production use would require verified data sources, governance, authentication and stronger validation.

12 Future Enhancement

The improvements below are proposed extensions only; none should be confused with already implemented features.

Priority	Proposed enhancement	Expected benefit
High	Authorized database / CSV / Excel ingestion	Replace seeded demonstration records with traceable data
High	Authentication and role-based access	Protect student-level personal data
High	Input schema + data quality validation	Detect missing and invalid real-world records
Medium	Daily or weekly attendance history	Enable genuine line charts and trends
Medium	Drill-through from aggregate charts to tables	Connect high-level observations to filtered rows
Medium	PDF reporting and scheduled summaries	Support periodic official communication
Medium	Responsive mobile layout QA	Improve usability on small screens
Low	Theme presets and accessible color modes	Improve readability across audiences

13 References

Primary project sources (supplied with the request):

- visualization-project-index.pdf - 14-section report structure and visualization headings.
- KVSR_Dashboard_Advanced-1(3).py - actual application logic, chart code, dataset generator and UI.

Technology documentation for further learning (reference URLs, not used to assert live project behavior):

- Python documentation - https://docs.python.org/3/

- Streamlit documentation - https://docs.streamlit.io/
- Pandas documentation - https://pandas.pydata.org/docs/
- Plotly Python documentation - https://plotly.com/python/

14 Appendix

Appendix A Source Code Overview

The source is a single-file application with these main blocks: (1) dependencies and Streamlit page configuration, (2) custom CSS, (3) cached generator, (4) chart and KPI helpers, (5) sidebar navigation and filters, and (6) individual page branches. The complete source is the separately supplied Python file; personal roster entries are not recopied here.

Representative computation steps (pseudocode)

```
df = create_data()           # seeded demo dataset
f  = filter_rows(df)         # department / year / gender / section / band
metrics = summarize(f)       # counts, averages, risk bands
figures = visualize(metrics, f) # bar, donut, sunburst, box, heatmap
render_streamlit_page(figures)
export_filtered_csv(f)
```

Appendix B Sample Dataset (Anonymized Schema)

Student ID	Department	Year	Section	Attendance
DEMO-001	CSE	1st Year	A	84.5%
DEMO-002	ECE	2nd Year	B	71.2%
DEMO-003	CSE (DS)	4th Year	A	92.1%

Appendix sample values above are illustrative and anonymized; they are not literal student rows from the supplied roster.

Appendix C Additional Screenshots

Four Android Chrome screenshots of the running localhost dashboard are included on the following pages. Screens show the interface, not a complete system acceptance test.

C.1 **Dashboard landing**
Smart Campus Intelligence welcome and demo-data disclosure.

C.2 **Executive KPI overview**
Student count, department count and sample attendance metrics.

C.3 **Navigation sidebar**
Seven dashboard views and the mobile navigation control.

C.4 **Smart Filters panel**
Department, year, gender, section and attendance-band selectors.

CHAPTER 14 / APPENDIX C - ADDITIONAL SCREENSHOTS

C.1 Campus Home Screen

LANDING PAGE / ANDROID CHROME

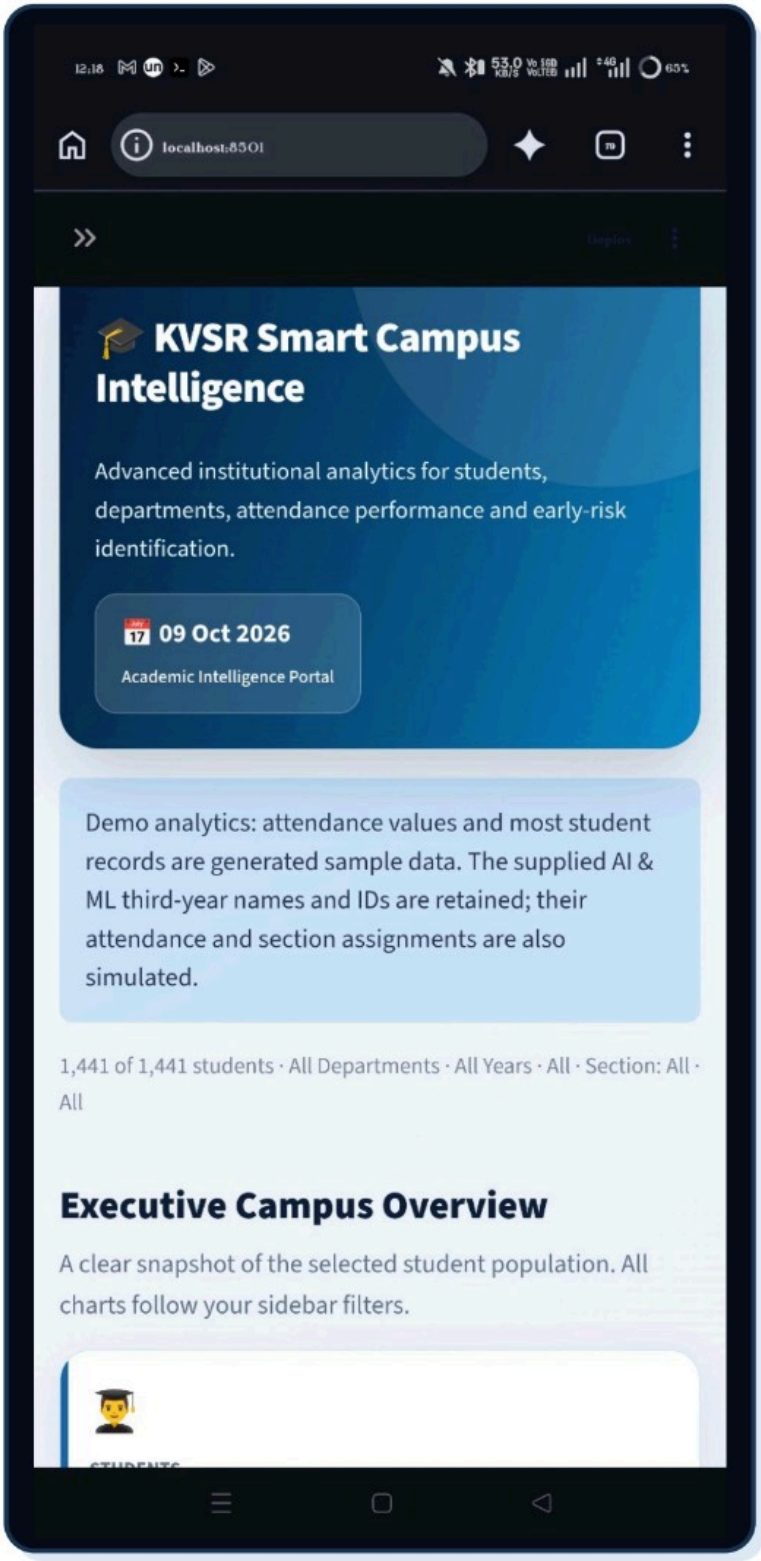


FIGURE C.1 / MOBILE UI

HOME AND PROJECT IDENTITY

The KVSR Smart Campus Intelligence landing page uses a prominent banner and a clear project introduction.

01 Project header

Gradient hero card with the KVSR Smart Campus Intelligence name.

02 Data notice

Visible disclaimer explains that attendance values and most records are demonstration data.

03 Browser access

The application is displayed inside mobile Chrome using localhost:8501.

EVIDENCE NOTE

User-supplied phone capture. Figures refer to demonstration data; not a device-performance test.

CHAPTER 14 / APPENDIX C - ADDITIONAL SCREENSHOTS

C.2 Executive KPI Overview

RESPONSIVE DASHBOARD / KPI CARDS

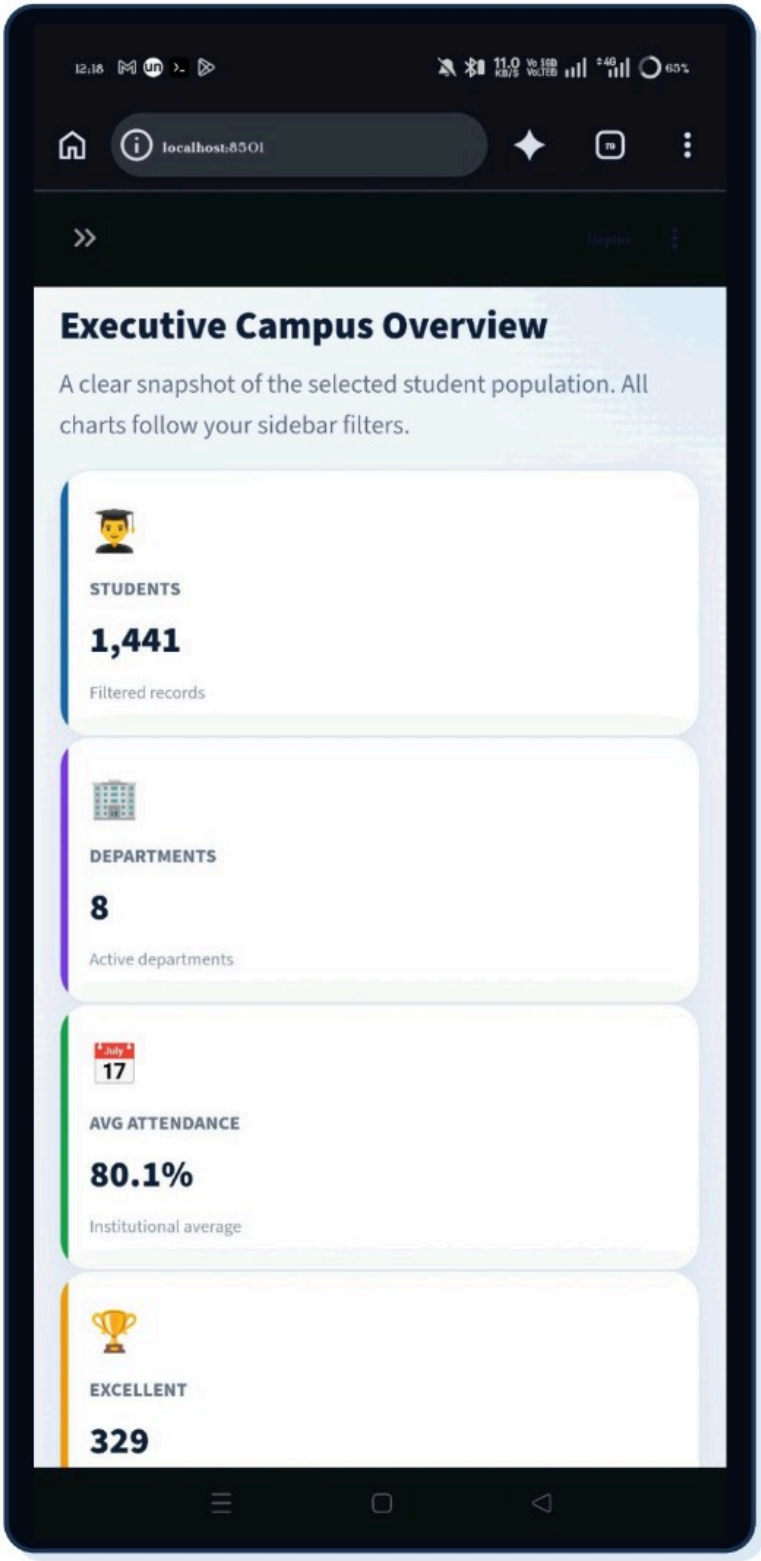


FIGURE C.2 / MOBILE UI

CAMPUS METRICS

The executive summary displays important metrics as separate stacked cards suitable for a narrow mobile screen.

01 Active students

1,441 records in the unfiltered demonstration dataset.

02 Campus coverage

Eight active departments are shown in the overview.

03 Attendance health

Average attendance is 80.1%; an excellent-attendance tile displays 329.

EVIDENCE NOTE

User-supplied phone capture. Figures refer to demonstration data; not a device-performance test.

CHAPTER 14 / APPENDIX C - ADDITIONAL SCREENSHOTS

C.3 Navigation Sidebar

NAVIGATION / MODULE ACCESS

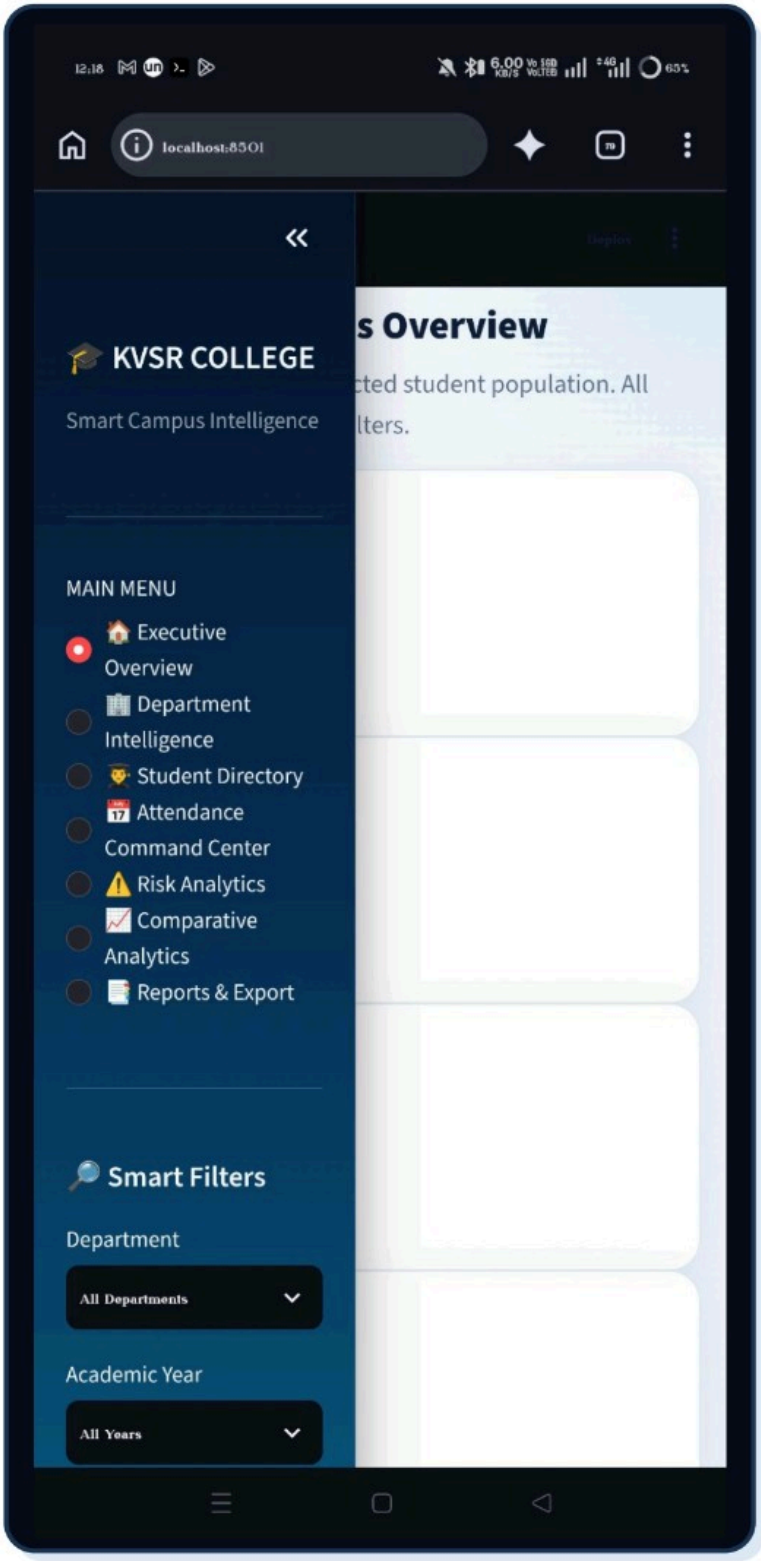


FIGURE C.3 / MOBILE UI

SEVEN DASHBOARD MODULES

The expanded sidebar gives users a single navigation point for the principal analytics pages.

01 Overview & departments

Executive Overview and Department Intelligence.

02 Student & attendance

Student Directory and Attendance Command Center.

03 Analysis & exports

Risk Analytics, Comparative Analytics, and Reports & Export.

EVIDENCE NOTE

User-supplied phone capture. Figures refer to demonstration data; not a device-performance test.

CHAPTER 14 / APPENDIX C - ADDITIONAL SCREENSHOTS

C.4 Smart Filter Controls

FILTERS / MOBILE INTERACTION

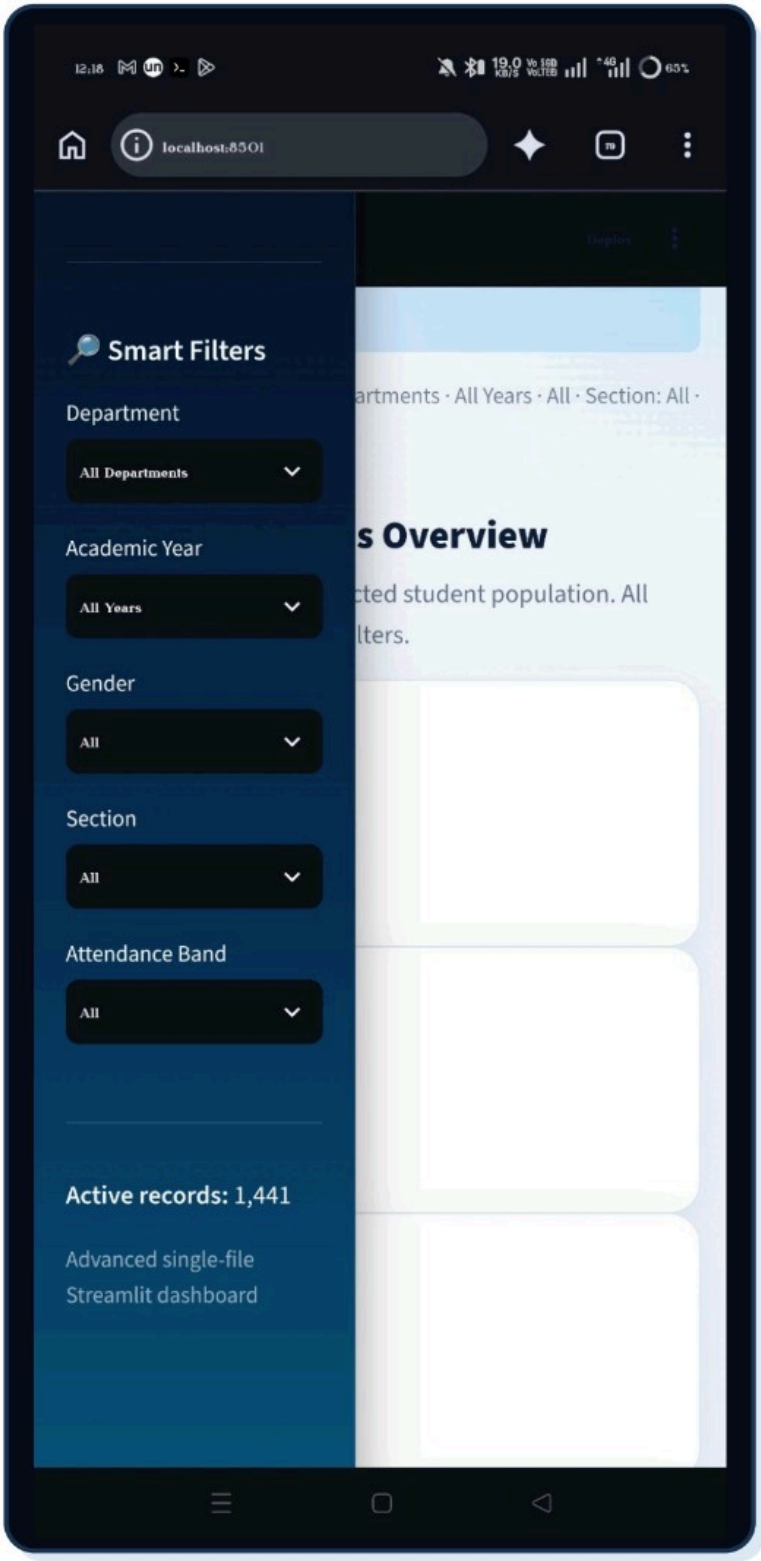


FIGURE C.4 / MOBILE UI

FILTERING INTERFACE

Dropdown inputs group the filters in a consistent sidebar, allowing exploration of subsets of students.

01 Population filters

Department, Academic Year, Gender and Section are visible.

02 Attendance grouping

Attendance Band selection is available.

03 Selection indicator

The sidebar shows 1,441 active records in this unfiltered example.

EVIDENCE NOTE

User-supplied phone capture. Figures refer to demonstration data; not a device-performance test.